OptiX device program logic

A visual walkthrough of cuda/DevicePrograms.cu

The device program renders one pixel by collecting the intersections of its camera ray with the original operand meshes. It then evaluates the STEP Boolean tree along that ray and shades the first boundary of the resulting solid.

The supplied file contains **BOX DIFFERENCE SPHERE**. The block spans 0 to 100 on each axis; the sphere has radius 40 and centre (80, 50, 50). The Boolean result exists as a ray state calculation. No final Boolean surface mesh is constructed.

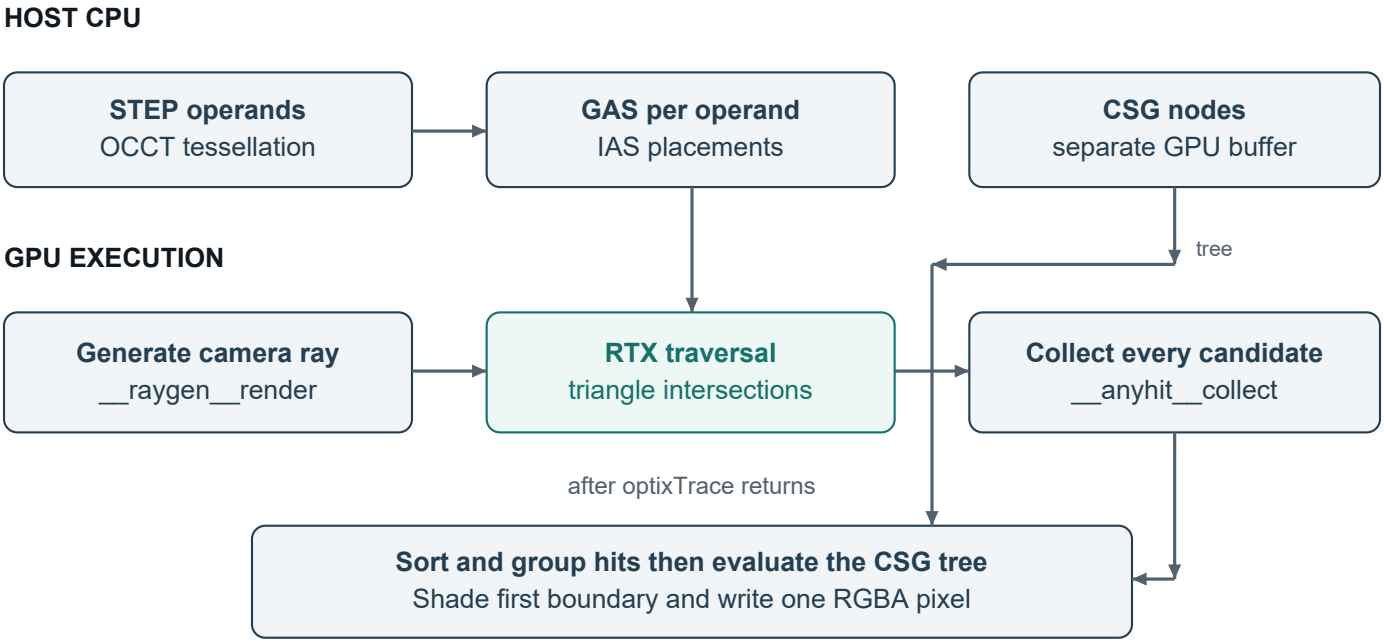


Figure 1. Spatial acceleration and Boolean semantics reach the device through separate inputs.

Why the ray tracing logic is CUDA code

OptiX device programs use CUDA C++ syntax. On the target RTX GPU, OptiX uses ray tracing hardware for acceleration-structure traversal and triangle intersection tests. Camera math, hit recording, sorting, Boolean evaluation and shading run as programmable GPU code. The call to `optixTrace` connects those parts.

Entry point	Role
__raygen__render	Generate a ray; trace, evaluate and write its pixel.
__anyhit__collect	Record each candidate hit and continue traversal.
__miss__all	Do nothing; recorded hits survive when traversal finishes.

GAS means geometry acceleration structure; IAS means instance acceleration structure. The host builds these, the pipeline and shader binding table in src/OptixRenderer.cpp, then invokes optixLaunch. There is no closest-hit program in this pipeline.

One pixel and all its intersections

Raygen reads the pixel index and samples its centre using the +0.5 offset. The horizontal coordinate includes aspect ratio; the vertical coordinate is inverted so the top image row points upward. The direction is normalized before tracing.

```
x = (2 * (pixel.x + 0.5) / width  - 1) * aspect * tanHalfFov
y = (1 - 2 * (pixel.y + 0.5) / height) * tanHalfFov
direction = unit(forward + right * x + up * y)
```

These formulas restate the helper calls in raygen. The ray starts at params.eye, uses t from 0 to 1e30, and disables closest-hit processing. Its normalized direction makes t a distance in model units. A local RayEvents buffer starts with count zero and overflow false.

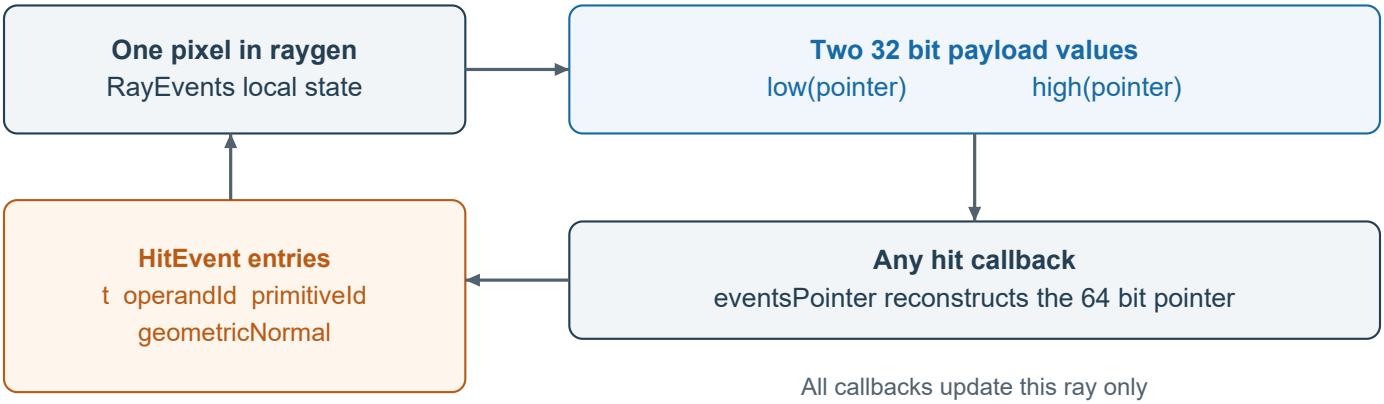


Figure 2. The payload carries an address, not the full hit array.

What the any hit callback records

The callback reconstructs the pointer from the two 32-bit payload values. It obtains the triangle index and the current instance ID. HitData in the shader binding table supplies that operand's vertex and index buffers. For the supplied file, operand 0 is the box and operand 1 is the sphere.

```
HitEvent = { t, operandId, primitiveId, geometricNormal }
```

The triangle normal comes from the cross product of its normalized edge directions. OptiX transforms it from object space to world space, and unit normalizes it again. Correct outward triangle winding is supplied by the OCCT mesh extraction.

`optixIgnoreIntersection()` rejects the candidate as a conventional accepted hit while retaining the event just recorded. Traversal continues without shortening the ray to that hit. Callback order is not assumed to be distance order.

If another callback arrives after 32 entries have been stored, the code increments the global overflow-ray counter, marks this ray and terminates traversal. Raygen then writes magenta instead of using an incomplete event set.

DevicePrograms.cu: raygen line 149, payload reconstruction line 27, any hit line 129

Sort and classify the events

After optixTrace returns, sortEvents uses insertion sort by t, then operand ID and triangle ID for exact ties. The fixed 32-hit capacity keeps the sort small.

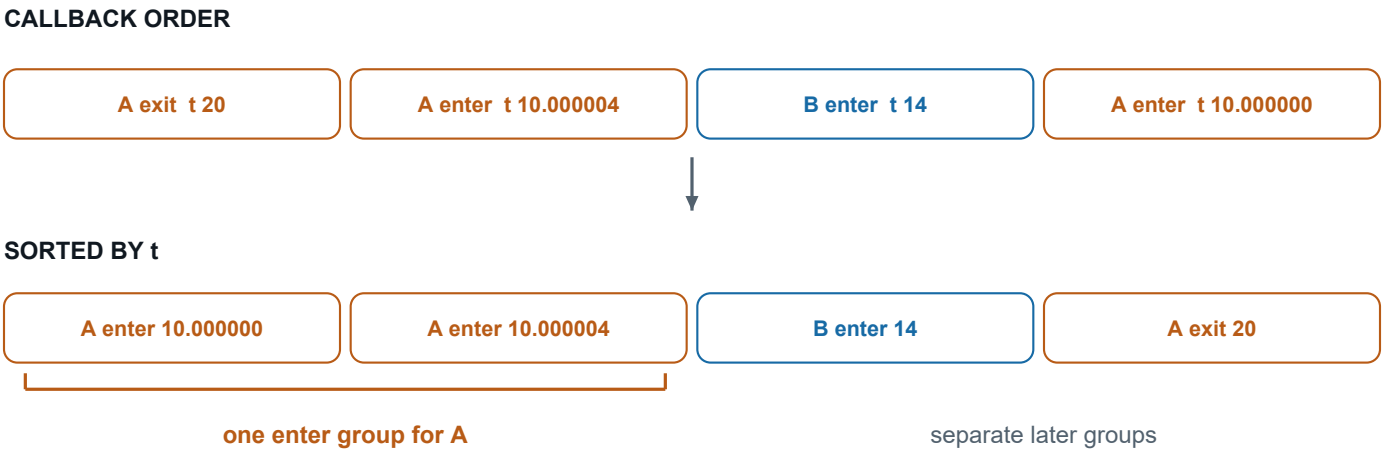


Figure 3. Illustrative hits form distance groups. Two box entries represent one crossing.

Group nearby distances

groupEnd compares each candidate with the first t in the group. Comparing only with successive neighbors could incorrectly grow the group beyond its tolerance.

```
epsilon = max(sceneEpsilon, 4 * FLT_EPSILON * abs(anchorT))
sceneEpsilon = 8 * FLT_EPSILON * sceneRadius
```

sceneRadius comes from the combined transformed operand bounds. This ray-distance tolerance is in model units and is separate from tessellation deflection.

Use the normal to identify a crossing

dot(direction, outward normal)	Sign mask	Meaning
Less than -8 * FLT_EPSILON	1	Enter the operand.
Greater than +8 * FLT_EPSILON	2	Leave the operand.
Magnitude at most 8 * FLT_EPSILON	0	Ignore a numerically grazing hit.
Enter and leave within one group	3	Keep the previous operand state.

groupSigns combines masks with bitwise OR: duplicate enters remain mask 1. Mask 3 is skipped as a tangent or a feature thinner than the grouping tolerance.

Initial state: visibleBoundary finds each operand's first mask 1 or 2. A first enter means the origin was outside; a first exit means it was inside. Without a valid crossing, the default is outside.

DevicePrograms.cu: sorting line 38, classification line 60, grouping line 66

Evaluate the Boolean state

Each operand has an inside flag. evaluate walks the GPU node array in postorder: a leaf reads its operand flag, and an operator combines its already evaluated children. The IAS contains spatial instances; it does not encode these Boolean operators.

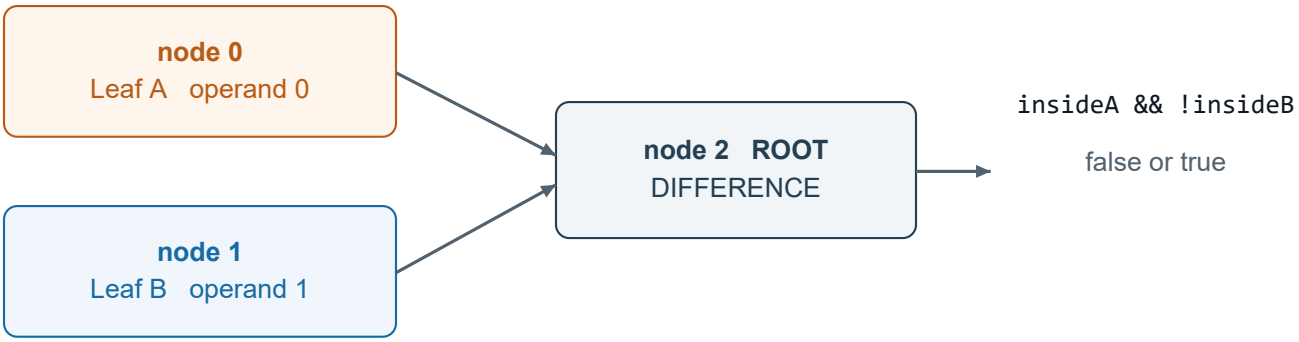


Figure 4. The supplied root evaluates the box flag AND NOT the sphere flag.

A	B	UNION A || B	INTERSECTION A && B	DIFFERENCE A && !B
0	0	0	0	0
0	1	1	0	0
1	0	1	0	1
1	1	1	1	0

A boundary is a change of result state

```
oldResult = evaluate(inside)
apply the valid crossings in this distance group
newResult = evaluate(inside)
if oldResult != newResult: return this group
otherwise: continue to the next group
```

The real loop updates operands in stable ID order and reevaluates after each update to find a contributor. It accepts a boundary only when the final group result differs from oldResult. Temporary changes within a coincident group do not create a surface.

groupSigns selects each operand's hit with the largest absolute normal-dot-direction value, then the lower triangle ID on a tie. The last operand in ID order that changes the result supplies the returned hit. This resolves coincident contributors deterministically.

Orient the normal for the resulting solid

A false-to-true transition requires a normal opposing the ray; true-to-false requires one pointing along it. visibleBoundary flips a disagreeing normal, including on subtraction cavity walls. For an origin inside the result, it returns the first exit.

DevicePrograms.cu: tree evaluation line 46, representative selection line 74, state sweep line 86

A ray through the supplied cavity

Consider an explanatory ray from (150, 50, 50) in direction (-1, 0, 0). It passes through the sphere centre and then through the box. This uses the supplied geometry; it is not the default camera ray.

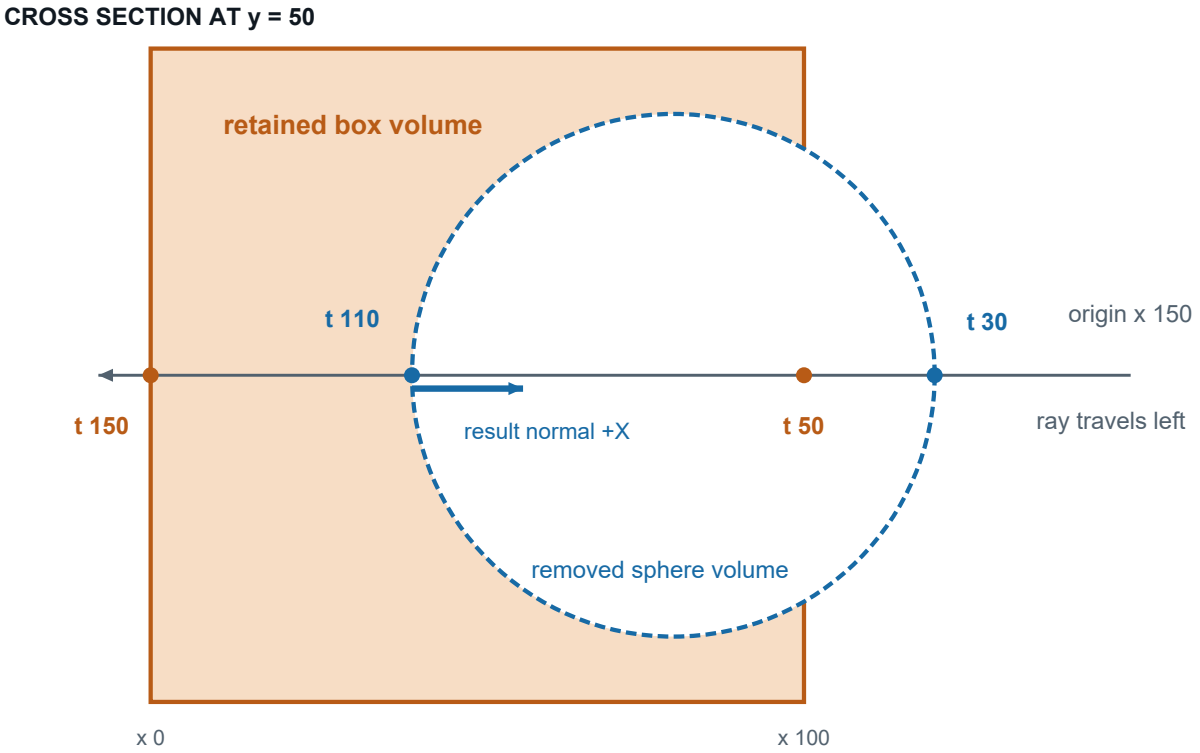


Figure 5. Ideal cross section of the original operands. This diagram is not renderer output; triangulation can shift sphere hit distances slightly.

t	Crossing	Inside A	Inside B	A && !B	Action
0	Ray origin	0	0	0	Start outside.
30	Enter sphere at x 120	0	1	0	Skip.
50	Enter box at x 100	1	1	0	Skip.
110	Exit sphere at x 40	1	0	1	Return blue cavity.
150	Exit box at x 0	0	0	0	Later boundary.

The nearest triangle hit is the sphere at t 30, but it does not bound retained material. The box entry at t 50 is also inside the removed sphere. At t 110 the ray leaves the sphere while remaining inside the box, so the result changes from false to true.

The sphere's outward normal there is (-1, 0, 0). The retained solid's cavity normal is (1, 0, 0), so visibleBoundary reverses it. All hits have already been collected, but the Boolean sweep returns at t 110 and does not shade the later box exit.

Shade the boundary and write the pixel

Raygen selects the base colour from the contributing operand type. It uses a dark background when visibleBoundary returns false. An empty miss callback does not mean the pixel must be background: all ignored candidates are still available for the CSG sweep.

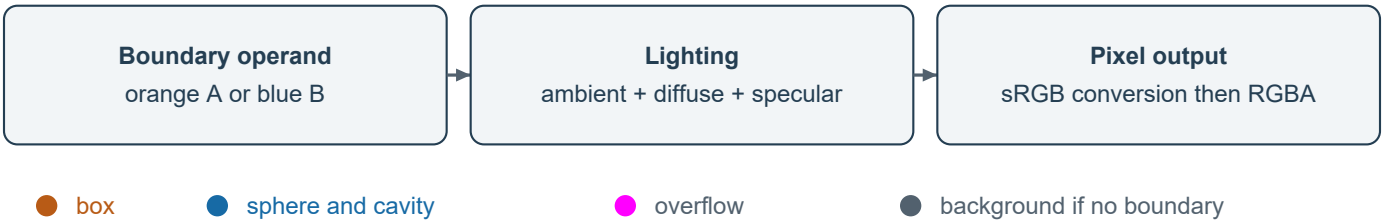


Figure 6. The selected operand controls the debug colour; lighting controls its brightness.

For lighting only, a normal facing away from the camera is flipped toward it. This makes inside views readable and is separate from the result-outward normal. The light is fixed relative to the camera; specular lighting adds a small Blinn-Phong highlight.

```
diffuse = max(0, dot(normal, light))
specular = diffuse > 0 ? 0.16 * max(0, dot(normal, halfway))^40 : 0
colour = base * (0.16 + 0.78 * diffuse) + specular
```

The ^40 denotes a power, implemented with powf. channel clamps colour to [0, 1], converts linear values to sRGB and rounds to 8 bits. The pixel index is pixel.y * width + pixel.x; alpha is 255. The host WIC writer encodes the PNG.

The small CUDA math helpers

Helper	Purpose
add sub mul	Vector addition, subtraction and multiplication by a scalar.
dot3	A scalar alignment measure, used for crossing signs and lighting.
cross3	A perpendicular vector, used to recover triangle normals.
unit	Normalize after scaling by the largest absolute component; return zero for a zero vector.

These vector helpers support the OptiX programs. optixTrace requests the accelerated intersection work.

Scope and source locations

Capacity is 32 raw hits per ray, 16 operands and 63 nodes. Grouping can collapse thin intervals. The result follows the tessellated operands, which must be closed and wound outward. Each pixel uses one opaque primary-ray boundary; there are no secondary rays, shadows or reflections.

Source snapshot: 15 September 2026. Algorithm: cuda/DevicePrograms.cu. Data: cuda/DeviceShared.h and src/CsgTypes.h. Host setup: src/OptixRenderer.cpp. Geometry: INPUT/box_sphere_intersection.STEP.

This is a source walkthrough, not a measured render or build validation. The renderer was not compiled or executed to make the guide.